



Sri Lanka Institute of Information Technology

IT3021 - Data warehousing and Business Intelligence

Assignment 01

Dahanayake V O

IT23302044

DS.WE.Y3.S1.01.02

Table of Contents

Table of Contents	1
Task 1 — Dataset Selection	3
Task 2 — Data Sources Preparation	5
Task 3 — Solution Architecture	9
Task 4 — Data Warehouse Design & Development	11
Task 5 — ETL Development.....	15
Task 6 — Accumulating Fact Table.....	34

Task 1 — Dataset Selection

1. Dataset Selection

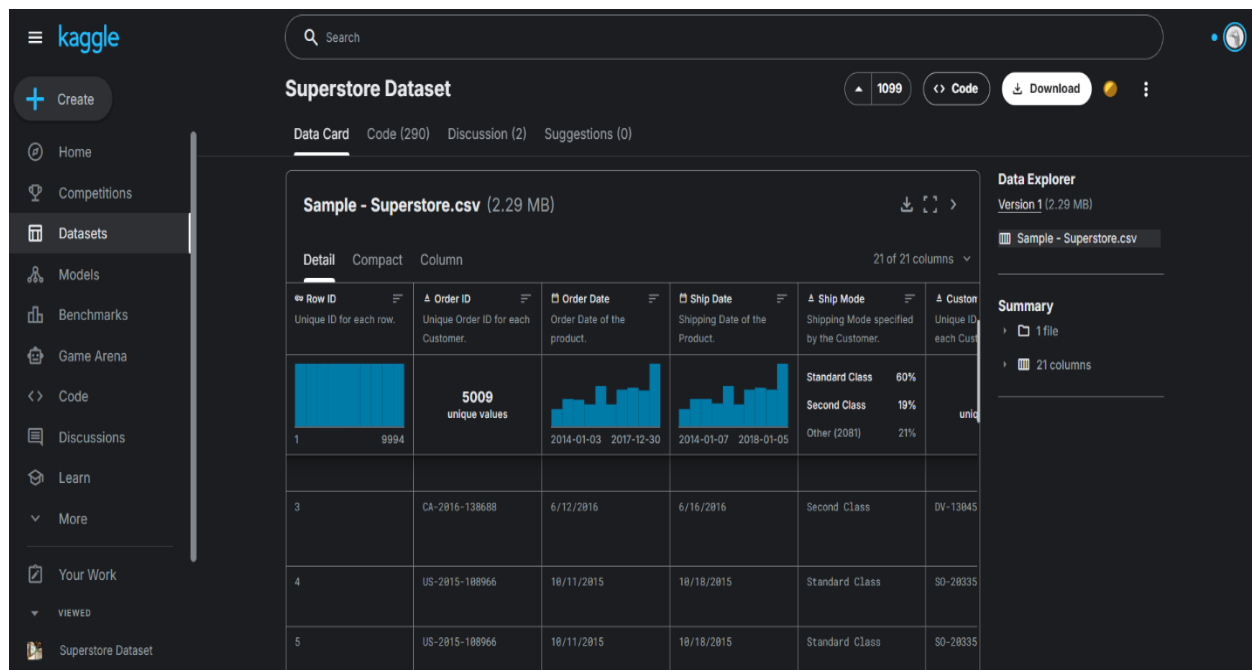
Dataset Name: Superstore Sales Dataset

Source: Kaggle — <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

Description:

The selected dataset is the Superstore Sales Dataset, which represents raw transactional sales data from a fictional retail superstore operating across the United States. The dataset is an OLTP-style dataset containing individual order transactions recorded between 2019 and 2022, covering approximately 4 years of business activity.

The dataset contains 9,994 order records with 21 attributes covering customer information, product details, geographical data, and sales metrics. It represents a realistic retail business scenario where customers place orders for products belonging to different categories and subcategories, delivered via different shipping modes.



DWBI ASSIGNMENT I – IT23302044

File

Home

Insert

Draw

Page Layout

Formulas

Data

Review

View

Developer

Help

Undo

Redo

Clipboard

Font

Paragraph

Alignment

Number

Styles

Conditional Formatting

Format as Table

Cell Styles

Insert

Delete

Format

Filter

Sort & Select

Find & Select

Analyze Data

Sensitivity

W23

▼

fx

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
1	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer	Customer Segment	Country	City	State	Postal Cod	Region	Product ID	Category	Sub-Categ	Product Name	Sales	Quantity	Discount	Profit		
2	1	CA-2016-1	11/8/2016	11/11/2016	Second Cl	CG-12520	Claire Gut	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-1F	Furniture	Bookcases	Bush Somerset Col	261.96	2	0	41.9136	
3	2	CA-2016-1	11/8/2016	11/11/2016	Second Cl	CG-12520	Claire Gut	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-1F	Furniture	Chairs	Hon Deluxe Fabric	731.94	3	0	219.582	
4	3	CA-2016-1	6/12/2016	6/16/2016	Second Cl	DV-13045	Darrin Van	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-1O	Office Sup	Labels	Self-Adhesive Addr	14.62	2	0	6.8714	
5	4	US-2015-1	10/11/2015	10/18/2015	Standard C	SO-20335	Sean O'Do	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-1C	Furniture	Tables	Bretford CR4500 S	957.5775	5	0.45	-383.031	
6	5	US-2015-1	10/11/2015	10/18/2015	Standard C	SO-20335	Sean O'Do	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-1O	Office Sup	Storage	Eldon Fold 'N Roll	22.368	2	0.2	2.5164	
7	6	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hc	Consumer	United States	Los Angeles	California	90032	West	FUR-FU-1C	Furniture	Furnishings	Eldon Expressions	48.86	7	0	14.1694	
8	7	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hc	Consumer	United States	Los Angeles	California	90032	West	OFF-AR-1C	Office Sup	Art	Newell 322	7.28	4	0	1.9656	
9	8	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hc	Consumer	United States	Los Angeles	California	90032	West	TEC-PH-1C	Technolog	Phones	Mitel 5320 IP Phor	907.152	6	0.2	90.7152	
10	9	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hc	Consumer	United States	Los Angeles	California	90032	West	OFF-BI-1O	Office Sup	Binders	DXL Angle-View Bi	18.504	3	0.2	5.7825	
11	10	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hc	Consumer	United States	Los Angeles	California	90032	West	OFF-AP-1O	Office Sup	Appliances	Belkin F5C206VTEI	114.9	5	0	34.47	
12	11	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hc	Consumer	United States	Los Angeles	California	90032	West	FUR-TA-1C	Furniture	Tables	Chromcraft Rectar	1706.184	9	0.2	85.3092	
13	12	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hc	Consumer	United States	Los Angeles	California	90032	West	FUR-TA-1C	Furniture	Tables	Konftel 250 Confe	911.424	4	0.2	68.3568	
14	13	CA-2017-1	4/15/2017	4/20/2017	Standard C	AA-10480	Andrew Al	Consumer	United States	Concord	North Carolina	28027	South	OFF-PA-1O	Office Sup	Paper	Xerox 1967	15.552	3	0.2	5.4432	
15	14	US-2016-1	12/5/2016	12/10/2016	Standard C	IM-15070	Irene Mad	Consumer	United States	Seattle	Washington	98103	West	OFF-BI-1O	Office Sup	Binders	Fellowes PB200 Pl	407.976	3	0.2	132.5922	
16	15	US-2015-1	11/22/2015	11/26/2015	Standard C	HP-14815	Harold Pav	Home Offi	United States	Fort Worth	Texas	76106	Central	OFF-AP-1O	Office Sup	Appliances	Holmes Replacem	68.81	5	0.8	-123.858	
17	16	US-2015-1	11/22/2015	11/26/2015	Standard C	HP-14815	Harold Pav	Home Offi	United States	Fort Worth	Texas	76106	Central	OFF-BI-1O	Office Sup	Binders	Storx DuraTech R	2.544	3	0.8	-3.816	
18	17	CA-2014-1	5/13/2014	5/15/2014	Standard C	AG-10270	Alejandro	Consumer	United States	West Jord	Utah	84084	West	OFF-ST-1O	Office Sup	Storage	Fellowes Super Stc	55.5	2	0	9.99	
19	18	CA-2014-1	8/27/2014	9/1/2014	Second Cl	ZD-21925	Zuschuss C	Consumer	United States	San Franci	California	94109	West	OFF-AR-1C	Office Sup	Art	Newell 341	8.56	2	0	2.4824	
20	19	CA-2014-1	8/27/2014	9/1/2014	Second Cl	ZD-21925	Zuschuss C	Consumer	United States	San Franci	California	94109	West	TEC-PH-1C	Technolog	Phones	Cisco SPA 501G IP	213.48	3	0.2	16.011	
21	20	CA-2014-1	8/27/2014	9/1/2014	Second Cl	ZD-21925	Zuschuss C	Consumer	United States	San Franci	California	94109	West	OFF-BI-1O	Office Sup	Binders	Wilson Jones Hang	22.72	4	0.2	7.384	
22	21	CA-2014-1	8/27/2014	9/1/2014	Second Cl	ZD-21925	Zuschuss C	Consumer	United States	San Franci	California	94109	West	OFF-AP-1O	Office Sup	Appliances	Acco Six-Outlet Po	60.34	7	0	15.6884	
23	22	CA-2016-1	12/9/2016	12/13/2016	Standard C	KB-16585	Ken Black	Corporate	United States	Fremont	Nebraska	68025	Central	OFF-AR-1C	Office Sup	Art	Newell 318	19.46	7	0	5.0596	
24	23	CA-2016-1	12/9/2016	12/13/2016	Standard C	KB-16585	Ken Black	Corporate	United States	Fremont	Nebraska	68025	Central	OFF-AP-1O	Office Sup	Appliances	Acco Six-Outlet Po	60.34	7	0	15.6884	
25	24	US-2017-1	7/16/2017	7/18/2017	Second Cl	SF-20065	Sandra Fla	Consumer	United States	Philadelph	Pennsylvania	19140	East	FUR-CH-1F	Furniture	Chairs	Global Deluxe Stac	71.372	2	0.3	-1.0196	
26	25	CA-2015-1	9/25/2015	9/30/2015	Standard C	EB-13870	Emily Burn	Consumer	United States	Orem	Utah	84057	West	FUR-TA-1C	Furniture	Tables	Bretford CR4500 S	1044.63	3	0	240.2649	
27	26	CA-2016-1	1/16/2016	1/20/2016	Second Cl	EH-13945	Eric Hoffm	Consumer	United States	Los Angeles	California	90049	West	OFF-BI-1O	Office Sup	Binders	Wilson Jones Activ	11.648	2	0.2	4.2224	

Ready

Sample - Superstore

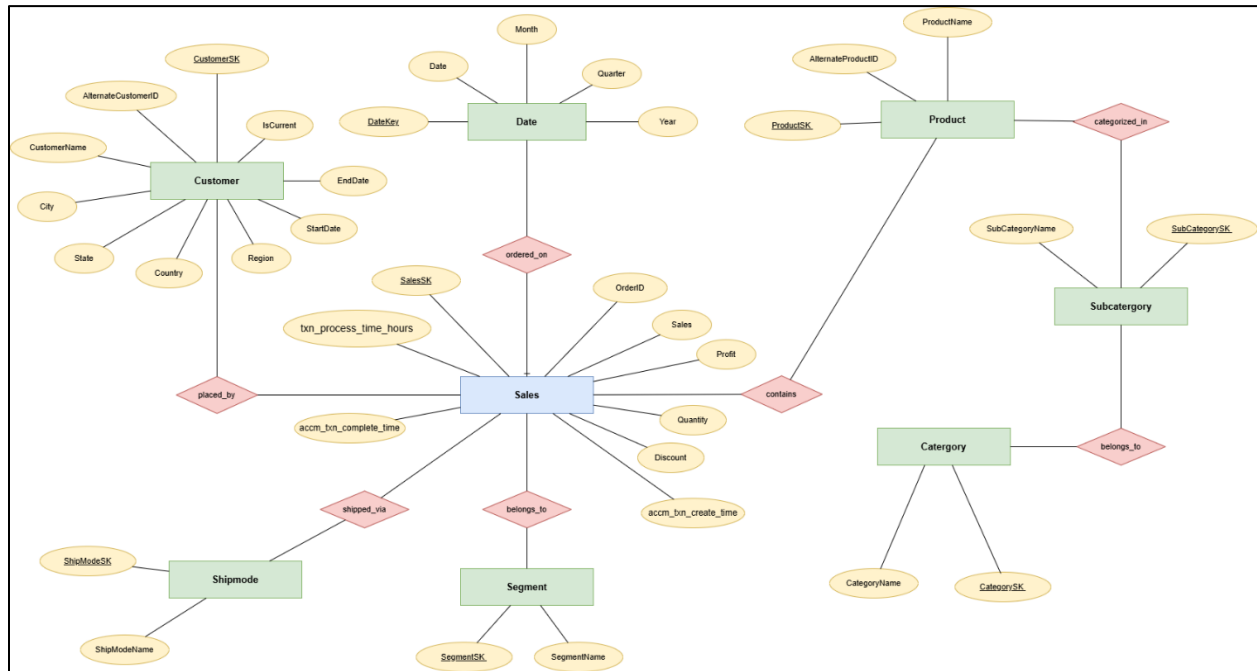
100%

Key Attributes:

Attribute	Description
Order ID	Unique identifier for each order
Order Date / Ship Date	Transaction and shipping dates
Customer ID / Name	Customer identification
Segment	Customer segment (Consumer, Corporate, Home Office)
Region / State / City	Geographic information
Product ID / Name	Product identification
Category / Sub-Category	Product hierarchy
Sales / Profit / Quantity / Discount	Business measures
Ship Mode	Delivery method

Justification:

This dataset was selected because it satisfies all assignment requirements — it is an OLTP transactional dataset (not a pre-built OLAP dataset), covers more than one year of data, contains multiple entities with clear relationships, supports hierarchical analysis (Category → SubCategory → Product), and provides sufficient measures for analytical reporting.

ER Diagram :**Task 2 — Data Sources Preparation****Preparation of Data Sources**

To demonstrate multiple source types as required by the assignment, the Superstore dataset was split into four separate source files of different formats. This simulates a realistic enterprise scenario where data originates from different systems and file types.

Source	Type	Content	Rows
Superstore_SourceDB	SQL Server Database	Customer demographics	793
Superstore_Source.xlsx	Excel File	Order transactions	9,994
Products.csv	CSV File	Product catalogue	1,894
CustomerLocation.txt	Text File	Customer locations	793

Source 1 — SQL Server Database (Superstore_SourceDB)

The customer demographic information was loaded into a SQL Server database table. This simulates an internal CRM system where customer records are maintained.

Attribute	Detail
Database Name	Superstore_SourceDB
Table Name	Customers
Columns	CustomerID, CustomerName, Segment
Row Count	793 unique customers

The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the database hierarchy expanded to 'Superstore_SourceDB' > 'Tables' > 'dbo.Customers'. The right pane shows a query window with the following SQL query:

```
SELECT * FROM dbo.Customers
```

The query results are displayed in a table with the following columns: CustomerID, CustomerName, and Segment. The results show 19 rows of customer data, including their IDs, names, and segments.

	CustomerID	CustomerName	Segment
1	AA-10315	Alex Avila	Consumer
2	AA-10375	Allen Arnold	Consumer
3	AA-10480	Andrew Allen	Consumer
4	AA-10645	Anna Andreadi	Consumer
5	AB-10015	Aaron Bergman	Consumer
6	AB-10060	Adam Bellavance	Home Office
7	AB-10105	Adrian Barton	Consumer
8	AB-10150	Aimee Bixby	Consumer
9	AB-10165	Alan Barnes	Consumer
10	AB-10255	Alejandro Ballentine	Home Office
11	AB-10600	Ann Blume	Corporate
12	AC-10420	Alyssa Crouse	Corporate
13	AC-10450	Amy Cox	Consumer
14	AC-10615	Ann Chong	Corporate
15	AC-10660	Anna Chung	Consumer
16	AD-10180	Alan Dominguez	Home Office
17	AF-10870	Art Ferguson	Consumer
18	AF-10885	Art Foster	Consumer
19	AG-10270	Alejandro Grove	Consumer

The status bar at the bottom indicates: Query executed suc... | DESKTOP-0T1VSQE\MSSQLSERVER... | sa (52) | Superstore_SourceDB | 00:00:00 | Row: 1, Col: 1 | 793 rows

Source 2 — Excel File (Superstore_Source.xlsx)

The order transaction data was kept in an Excel workbook. This simulates a scenario where daily sales transactions are exported from a POS system into Excel.

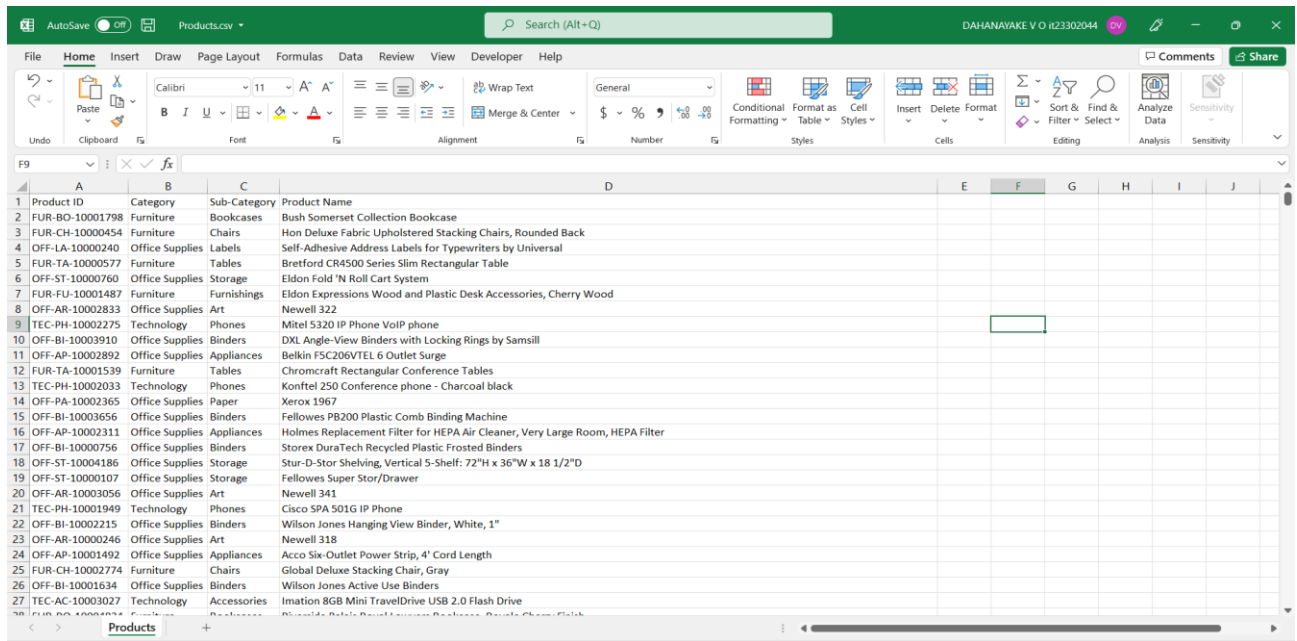
Attribute	Detail
File Name	Superstore_Source.xlsx
Sheet Name	Orders
Columns	Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Product ID, Sales, Quantity, Discount, Profit
Row Count	9,994 order records

Source 3 — CSV File (Products.csv)

Product details including category and subcategory hierarchy were saved as a CSV file. This simulates a product catalogue maintained by the merchandising team.

Attribute	Detail
File Name	Products.csv
Columns	Product ID, Category, Sub-Category, Product Name
Row Count	1,894 unique products

DWBI ASSIGNMENT I – IT23302044



Source 4 — Text File (CustomerLocation.txt)

Customer address and location information was saved as a tab-delimited text file. This simulates address data maintained separately from the core customer database, updated when customers report location changes.

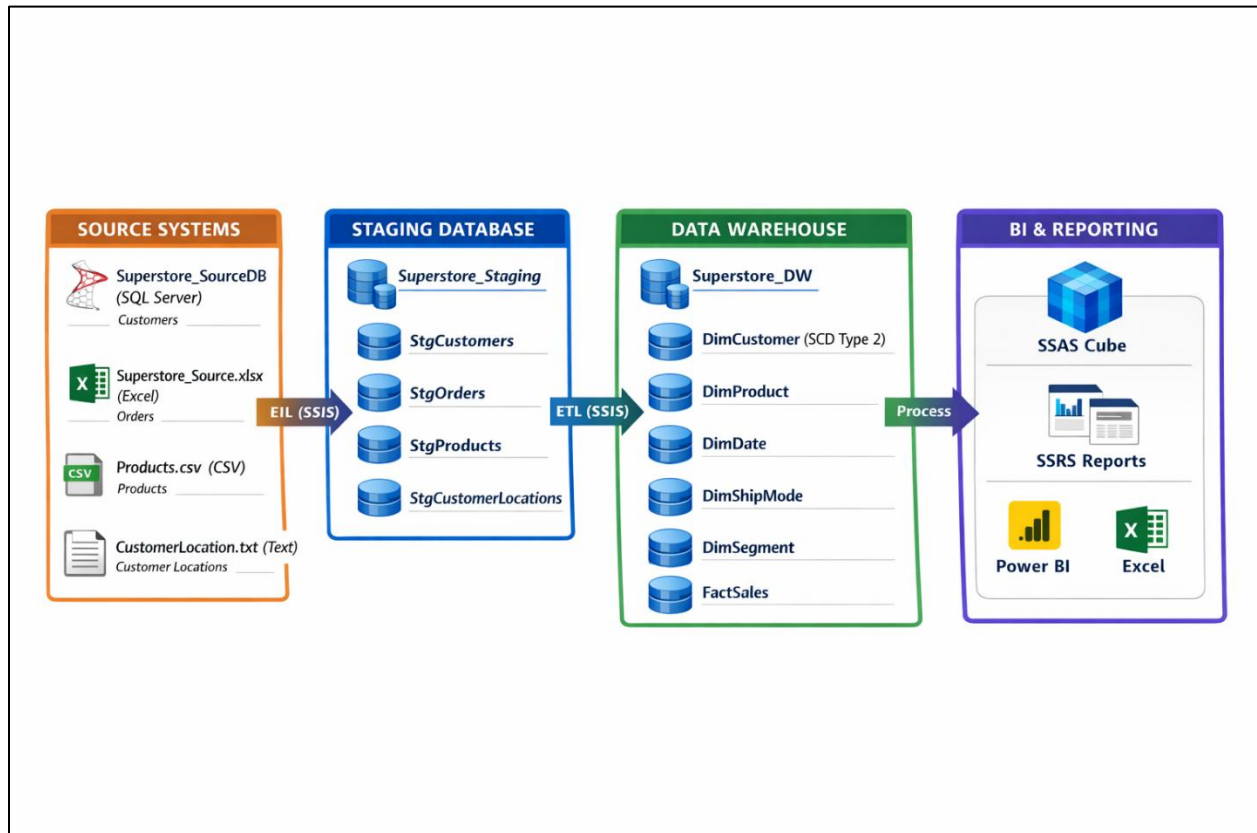
Attribute	Detail
File Name	CustomerLocation.txt
Format	Tab-delimited
Columns	Customer ID, City, State, Postal Code, Region, Country
Row Count	793 unique customer locations

Customer ID	Country	City	State	Postal Code	Region
CG-12520	United States	Henderson	Kentucky	42420	South
DV-13045	United States	Los Angeles	California	90036	West
SO-20335	United States	Fort Lauderdale	Florida	33311	South
BH-11710	United States	Los Angeles	California	90032	West
AA-10480	United States	Concord	North Carolina	28027	South
IM-15070	United States	Seattle	Washington	98103	West
HP-14815	United States	Fort Worth	Texas	76106	Central
PK-19075	United States	Madison	Wisconsin	53711	Central
AG-10270	United States	West Jordan	Utah	84084	West
ZD-21925	United States	San Francisco	California	94109	West
KB-16585	United States	Fremont	Nebraska	68025	Central
SF-20065	United States	Philadelphia	Pennsylvania	19140	East
EB-13870	United States	Orem	Utah	84057	West
EH-13945	United States	Los Angeles	California	90049	West
TB-21520	United States	Philadelphia	Pennsylvania	19140	East
MA-17560	United States	Houston	Texas	77095	Central
GH-14485	United States	Richardson	Texas	75080	Central
SH-20710	United States	Houston	Texas	77041	Central
LC-16930	United States	Naperville	Illinois	60540	Central
RA-19885	United States	Los Angeles	California	90049	West
ES-14080	United States	Melbourne	Florida	32935	South
OH-18715	United States	Fagan	Minnesota	55122	Central
PO-18865	United States	Westland	Michigan	48185	Central
LH-16900	United States	Dover	Delaware	19901	East
DP-13000	United States	New Albany	Indiana	47150	Central
JM-15265	United States	New York City	New York	10024	East
TB-21055	United States	Troy	New York	12180	East
KM-16720	United States	Los Angeles	California	90004	West
PS-18970	United States	Chicago	Illinois	60610	Central
BS-11590	United States	Gilbert	Arizona	85234	West
KD-16270	United States	Springfield	Virginia	22153	South
HM-14980	United States	New York City	New York	10009	East
JE-15745	United States	Memphis	Tennessee	38109	South
KB-16600	United States	Houston	Texas	77041	Central
SC-20770	United States	Decatur	Alabama	35601	South
DN-13690	United States	San Francisco	California	94122	West
JC-16105	United States	Durham	North Carolina	27707	South
CS-12400	United States	Chicago	Illinois	60623	Central
PG-18895	United States	Rochester	Minnesota	55901	Central

Task 3 — Solution Architecture

Solution Architecture

The following diagram illustrates the high-level Data Warehouse and Business Intelligence solution architecture designed for the Superstore dataset.



Component Descriptions:

Source Systems

The solution draws data from four different source types representing a realistic multi-source enterprise environment:

- **Superstore_SourceDB** — A SQL Server database containing customer demographic records
- **Superstore_Source.xlsx** — An Excel file containing daily order transaction data
- **Products.csv** — A CSV file containing the product catalogue with category hierarchy
- **CustomerLocation.txt** — A tab-delimited text file containing customer address information

Staging Layer (Superstore Staging)

The staging database acts as an intermediate landing zone between the source systems and the data warehouse. Data is extracted from all four sources and loaded into staging tables with minimal transformation. The staging tables mirror the structure of the source data, bringing all sources into a single unified SQL Server environment for further processing. The staging tables include StgCustomers, StgOrders, StgProducts and StgCustomerLocations.

ETL Layer (SSIS)

Two SSIS packages handle the ETL process:

- **Superstore_Load_Staging.dtsx** — Extracts data from all four source types and loads into the staging database
- **Superstore_Load_DW.dtsx** — Transforms and loads data from staging into the data warehouse, applying surrogate key generation, lookups, and Slowly Changing Dimension logic

Data Warehouse (Superstore DW)

The data warehouse follows a Snowflake Schema with one central fact table and seven dimension tables. Surrogate keys are used throughout to maintain referential integrity independently of source system IDs. The DimCustomer table implements SCD Type 2 to track historical address changes.

SSAS Cube

An SSAS (SQL Server Analysis Services) cube is built on top of the data warehouse, enabling multidimensional analysis with hierarchies, measures, and aggregations. The cube supports OLAP operations such as Roll-up, Drill-down, Slice, Dice and Pivot.

Reporting Layer

The BI reporting layer consists of:

- **Excel** — Connected to the SSAS cube for pivot table analysis and OLAP operations demonstration
- **Power BI** — Connected to the data warehouse for interactive dashboards, drill-down reports, drill-through reports and cascading slicer reports published to Power BI Service

Task 4 — Data Warehouse Design & Development

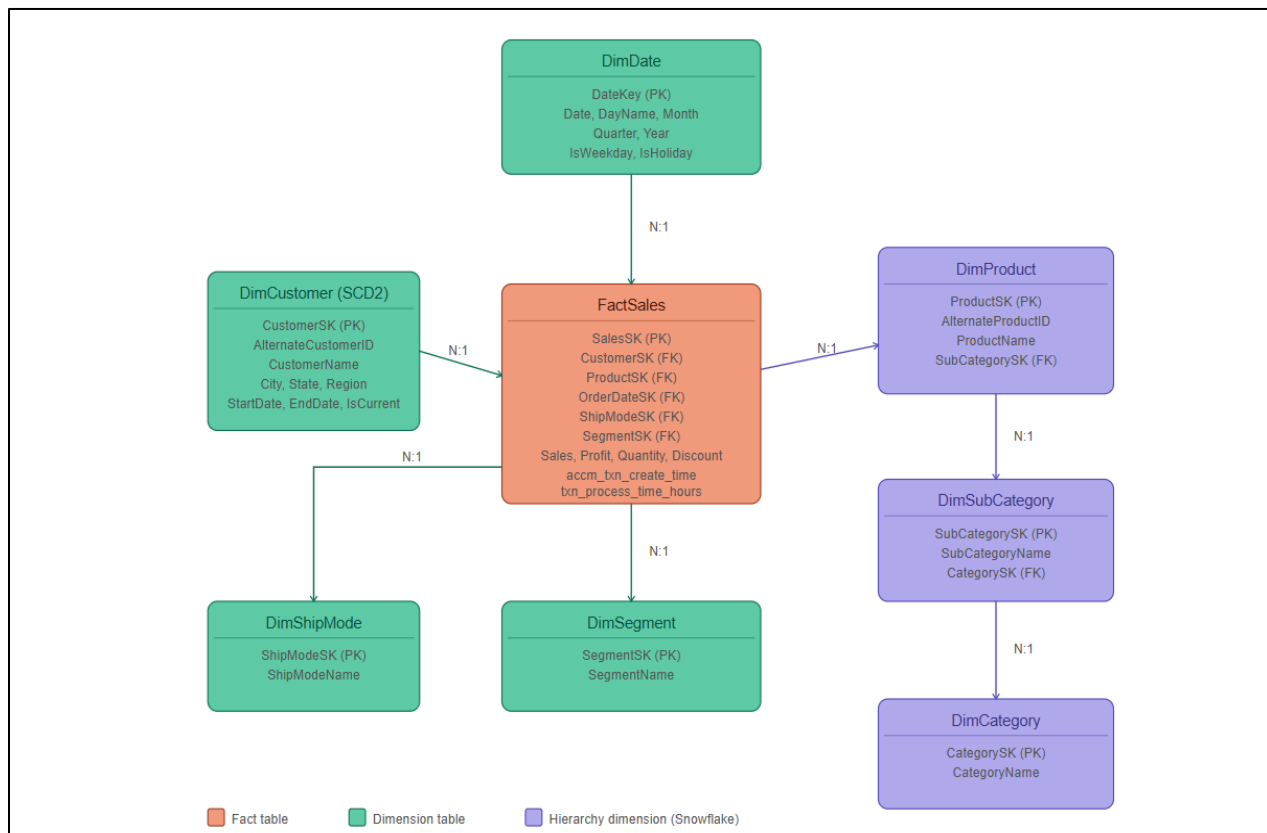
Data Warehouse Design and Development

Schema Selection

A **Snowflake Schema** was selected for the data warehouse design. This decision was made for the following reasons:

- The Superstore dataset has a natural three-level product hierarchy — Category → SubCategory → Product — which maps perfectly to a snowflake structure with separate dimension tables
- Snowflake schema reduces data redundancy by normalizing dimension tables — category names are not repeated for every product
- The assignment requires hierarchies for SSAS cube implementation — separate dimension tables make hierarchy definition cleaner and more maintainable
- The customer data naturally separates into demographic information and location information, which aligns with the snowflake approach.

Snowflake Schema Diagram



Dimensional Model

The data warehouse consists of **7 dimension tables** and **1 fact table** as described below.

Dimension Tables:

Table	Type	Description
DimCategory	Regular	Stores the 3 product categories
DimSubCategory	Regular	Stores 17 product subcategories linked to DimCategory
DimProduct	Regular	Stores 1,862 products linked to DimSubCategory
DimCustomer	SCD Type 2	Stores customer details with historical address tracking
DimShipMode	Regular	Stores 4 shipping modes
DimSegment	Regular	Stores 3 customer segments
DimDate	Regular	Auto-generated date dimension from DateMaster.sql

Fact Table:

Table	Grain	Measures
FactSales	One row per order line item	Sales, Profit, Quantity, Discount, accm_txn_create_time, accm_txn_complete_time, txn_process_time_hours

Slowly Changing Dimension

`DimCustomer` is implemented as a **SCD Type 2** dimension. Customer address attributes (City, State, PostalCode, Region, Country) are treated as **Historical attributes** — when a customer's address changes, a new record is inserted with a new `StartDate` and the old record's `EndDate` is updated. The `IsCurrent` flag identifies the latest active record.

Customer phone-related attributes are treated as **Changing attributes (Type 1)** — these are overwritten when changed as historical tracking is not required.

This design allows the business to answer questions such as "What was the customer's address when they placed that order in 2021?" — critical for accurate historical reporting.

SQL Implementation

All tables were created in SQL Server using the **Superstore_DW** database. The key design decisions in the SQL implementation include:

- All dimension tables use **IDENTITY(1,1)** for surrogate key auto-generation
- All text columns use **NVARCHAR** to support unicode characters
- **FactSales** contains a computed column **txn_process_time_hours** calculated as **DATEDIFF (HOUR, accm_txn_create_time, accm_txn_complete_time)**
- Foreign key relationships link **FactSales** to all dimension tables

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Superstore_DW' database is expanded, showing a hierarchy of objects: Database Diagrams, Tables, System Tables, FileTables, External Tables, Graph Tables, and various dimension tables (dbo.DimCategory, dbo.DimCustomer, dbo.DimDate, dbo.DimProduct, dbo.DimSegment, dbo.DimShipMode, dbo.DimSubCategory, dbo.FactSales), along with Views, External Resources, Synonyms, Programmability, Query Store, Service Broker, Storage, and Security.

In the center, a SQL query is shown in a text editor window:

```

129
130 USE Superstore_DW;
131
132 SELECT TABLE_NAME
133 FROM INFORMATION_SCHEMA.TABLES
134 WHERE TABLE_TYPE = 'BASE TABLE'
135 ORDER BY TABLE_NAME;

```

Below the query editor, the 'Results' tab is active, displaying the output of the query as a table:

	TABLE_NAME
1	DimCategory
2	DimCustomer
3	DimDate
4	DimProduct
5	DimSegment
6	DimShipMode
7	DimSubCategory
8	FactSales

Data Warehouse Row Counts After Loading

Table	Row Count
DimCategory	3
DimSubCategory	17
DimProduct	1,862
DimShipMode	4
DimSegment	3
DimCustomer	793
DimDate	~39,000
FactSales	9,986

The screenshot displays the SQL Server Enterprise Manager interface. The query window shows a script titled 'DWBI_Script...W (sa (71))' with the following SQL code:

```

USE Superstore_DW;

SELECT 'DimCustomer' AS TableName, COUNT(*) AS 'RowCount' FROM dbo.DimCustomer
UNION ALL
SELECT 'FactSales', COUNT(*) FROM dbo.FactSales;

USE Superstore_DW;

SELECT 'DimCategory' AS TableName, COUNT(*) AS 'RowCount' FROM dbo.DimCategory
UNION ALL
SELECT 'DimSubCategory', COUNT(*) FROM dbo.DimSubCategory
UNION ALL
SELECT 'DimProduct', COUNT(*) FROM dbo.DimProduct
UNION ALL
SELECT 'DimShipMode', COUNT(*) FROM dbo.DimShipMode
UNION ALL
SELECT 'DimSegment', COUNT(*) FROM dbo.DimSegment
UNION ALL
SELECT 'DimCustomer', COUNT(*) FROM dbo.DimCustomer
UNION ALL
SELECT 'FactSales', COUNT(*) FROM dbo.FactSales;

```

The Results pane at the bottom shows the output of the query, which matches the data in the table above:

TableName	RowCount
DimCategory	3
DimSubCategory	17
DimProduct	1862
DimShipMode	4
DimSegment	3
DimCustomer	793
FactSales	9986

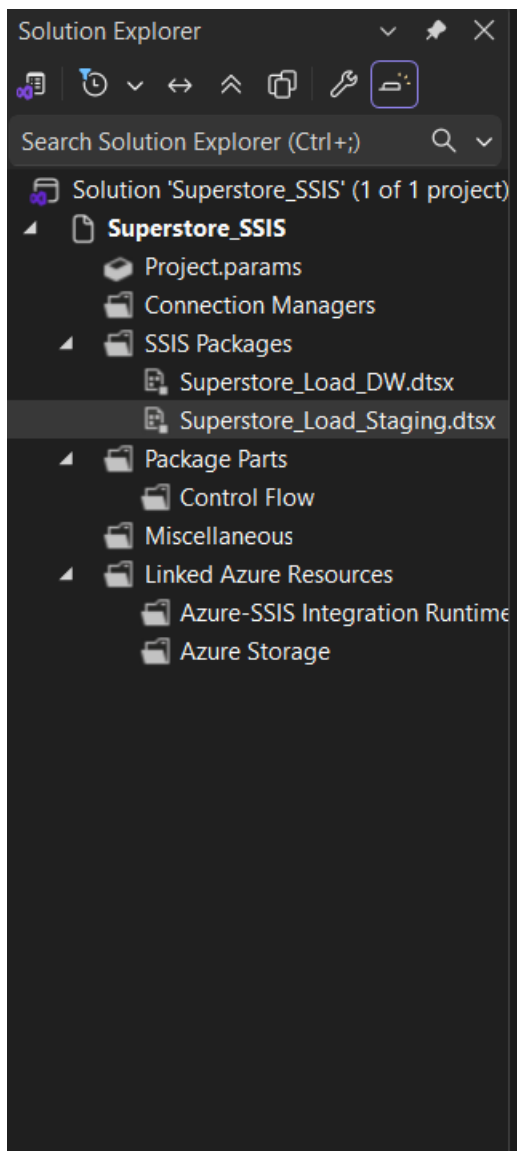
The status bar at the bottom indicates: Query executed successfully. | DESKTOP-0T1VSQE\MSSQLSERVER... | sa (71) | Superstore_DW | 00:00:00 | Row: 1, Col: 1 | 7 rows

Task 5 — ETL Development

ETL Development

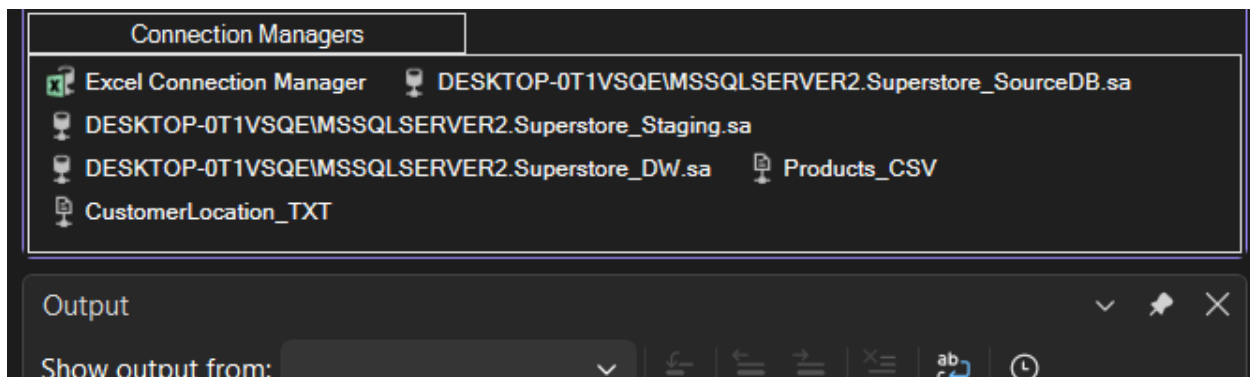
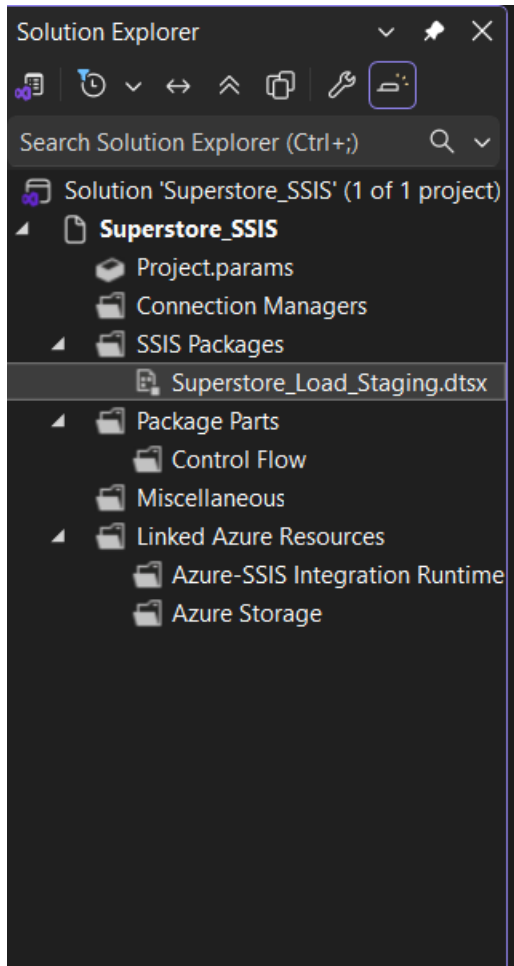
The ETL process was developed using SQL Server Integration Services (SSIS) and consists of two packages:

- **Superstore_Load_Staging.dtsx** — Extracts data from all 4 source types into the Staging database
- **Superstore_Load_DW.dtsx** — Transforms and loads data from Staging into the Data Warehouse



Package 1 — Superstore_Load_Staging.dtsx

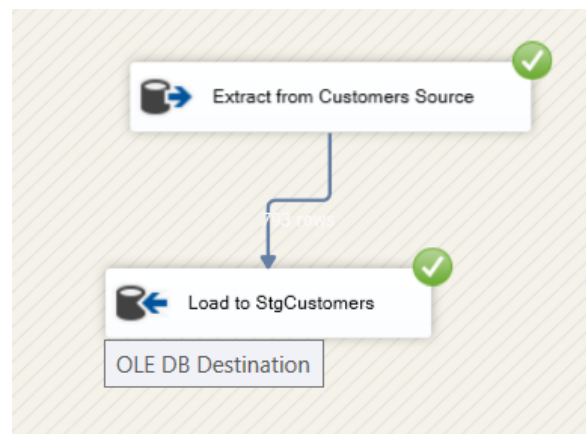
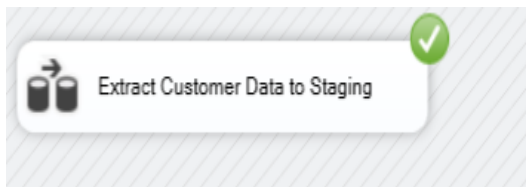
This package extracts data from all four source systems and loads them into the staging database with minimal transformation. Each source table has a corresponding staging table.



The package contains 4 Data Flow Tasks connected in serial order:

Task 1 — Extract Customer Data to Staging

- **Source:** Superstore_SourceDB (SQL Server database)
- **Destination:** StgCustomers in Superstore_Staging
- **Rows loaded:** 793
- An OnPreExecute Event Handler truncates StgCustomers before each load to prevent duplicates



Connection Manager

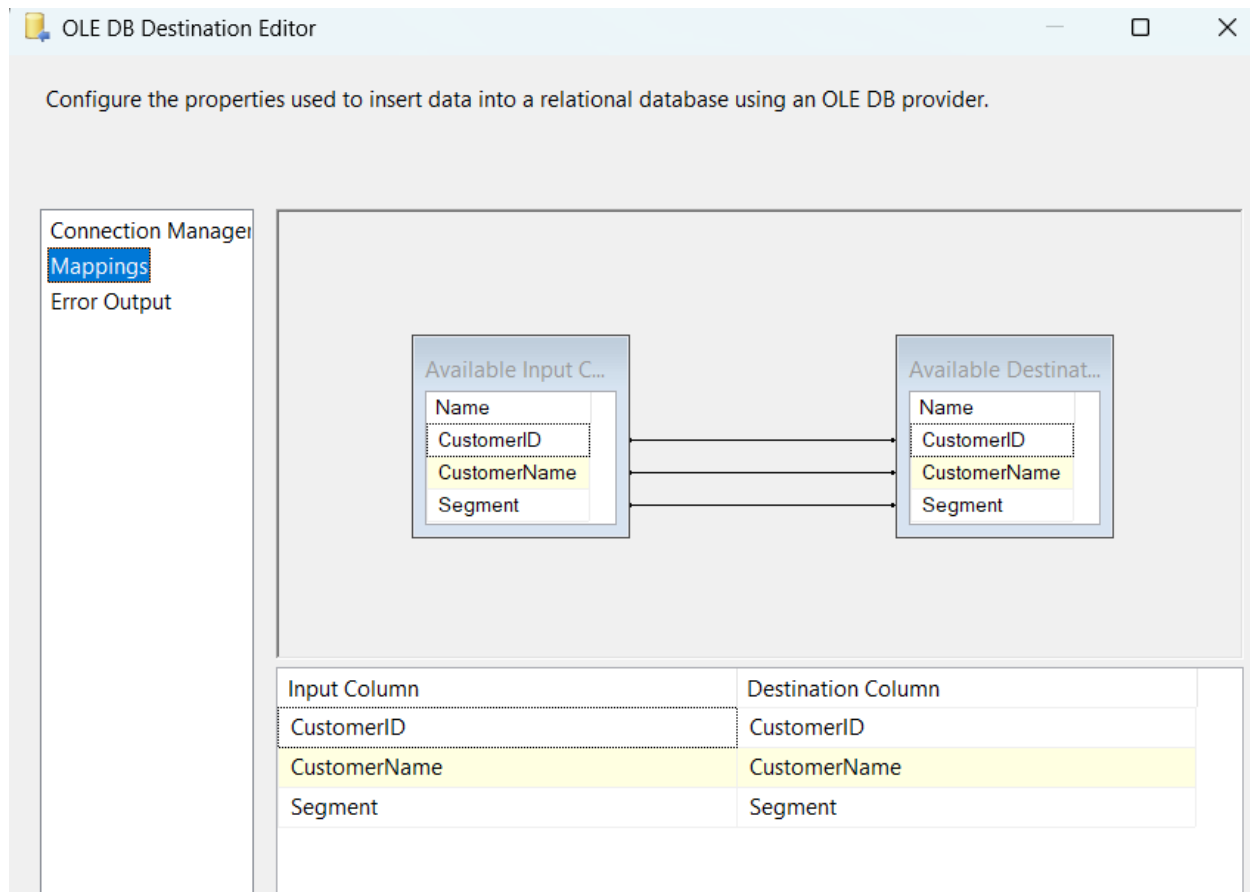
Columns

Error Output

Available External Col...

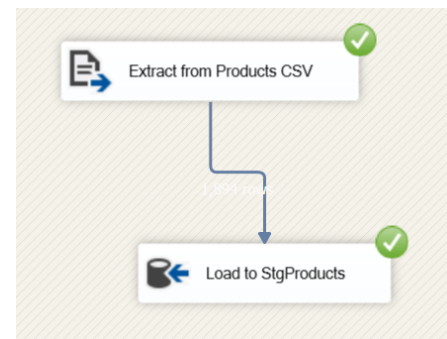
- ☒ Name
- ☒ CustomerID
- ☒ CustomerName

External Column	Output Column
CustomerID	CustomerID
CustomerName	CustomerName
Segment	Segment

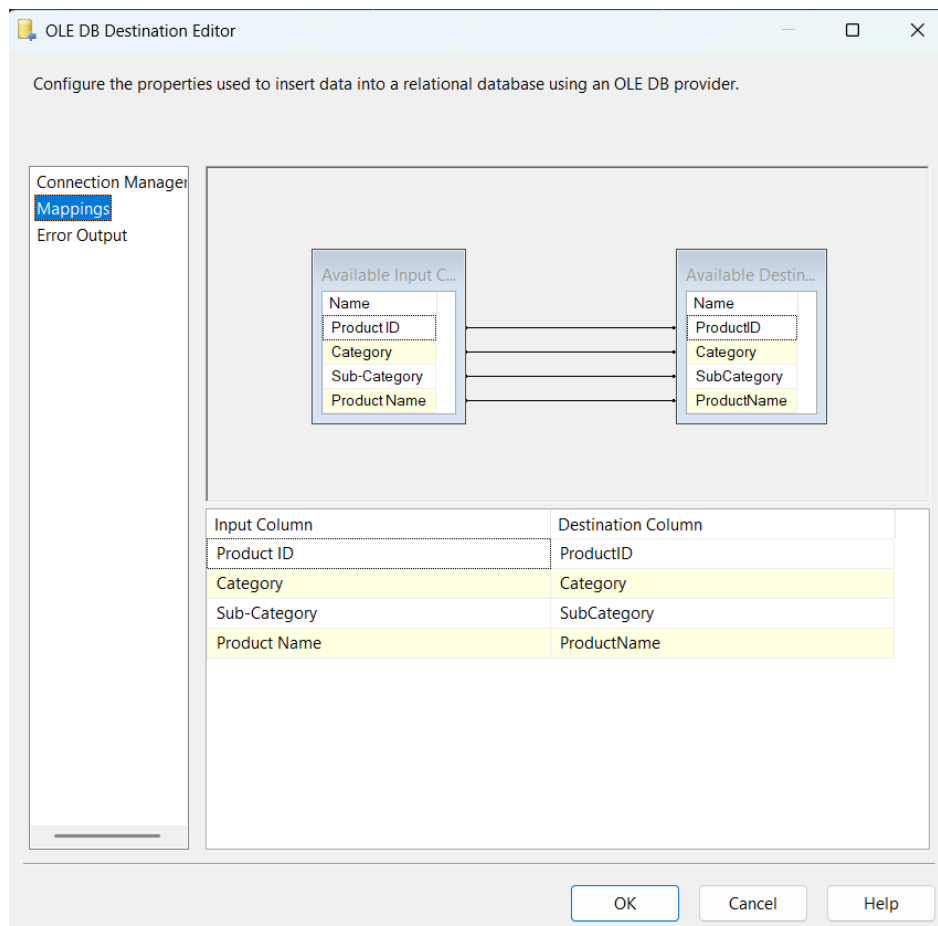
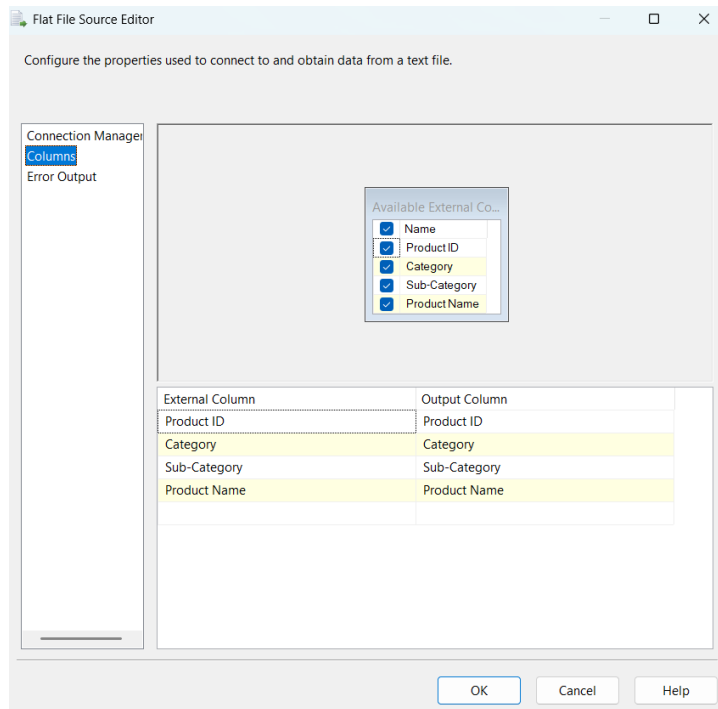


Task 2 — Extract Product Data to Staging

- Source: `Products.csv` (CSV flat file)
- Destination: `StgProducts` in `Superstore_Staging`
- Rows loaded: 1,894
- A Data Conversion component was used to handle unicode/non-unicode conversion from CSV source
- An OnPreExecute Event Handler truncates `StgProducts` before each load

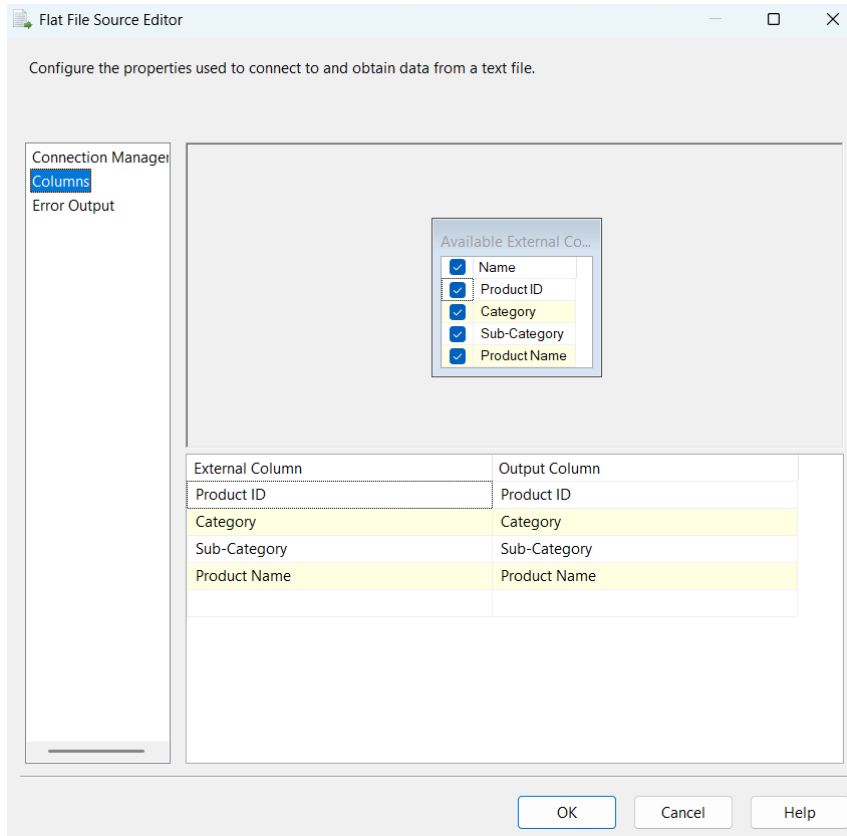
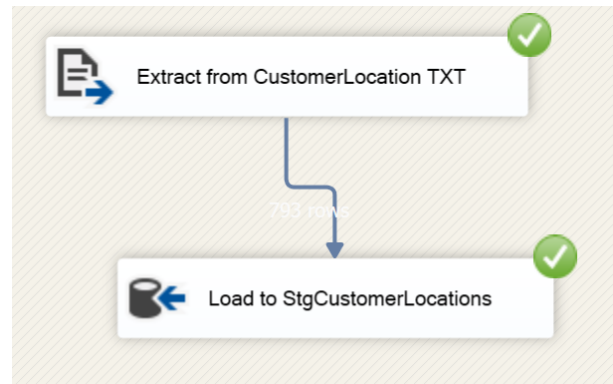
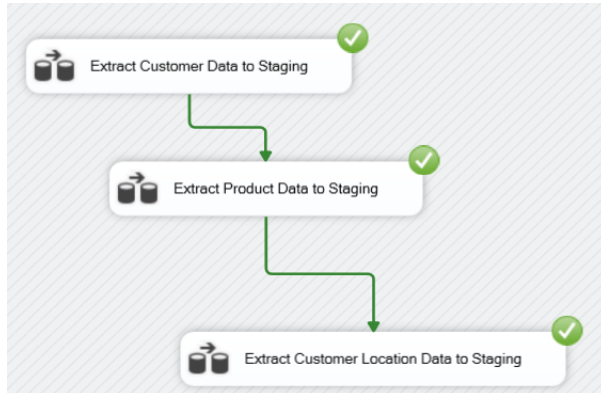


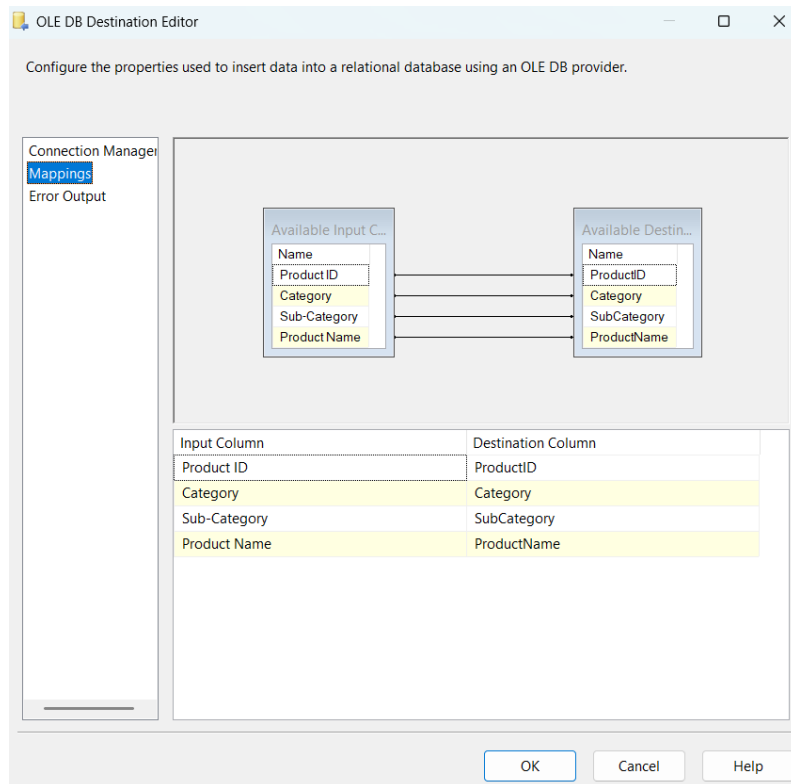
DWBI ASSIGNMENT I – IT23302044



Task 3 — Extract Customer Location Data to Staging

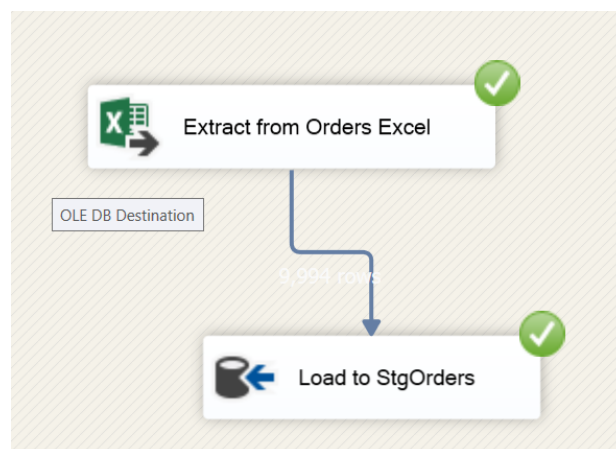
- **Source:** CustomerLocation.txt (Tab-delimited text file)
- **Destination:** StgCustomerLocations in Superstore_Staging
- **Rows loaded:** 793
- An OnPreExecute Event Handler truncates StgCustomerLocations before each load





Task 4 — Extract Order Data to Staging

- Source: Superstore_Source.xlsx (Excel file)
- Destination: StgOrders in Superstore_Staging
- Rows loaded: 9,994
- An OnPreExecute Event Handler truncates StgOrders before each load



DWBI ASSIGNMENT I – IT23302044

Excel Source Editor

Configure the properties that enable the Data Flow task to obtain data from Excel provider.

Connection Manager
Columns
Error Output

Available External ...

- ☒ Name
- ☒ Row ID
- ☒ Order ID
- ☒ Order Date
- ☒ Ship Date
- ☒ Ship Mode
- ☒ Customer ID
- ☒ Product ID
- ☒ Sales
- ☒ Quantity

External Column	Output Column
Row ID	Row ID
Order ID	Order ID
Order Date	Order Date
Ship Date	Ship Date
Ship Mode	Ship Mode
Customer ID	Customer ID
Product ID	Product ID
Sales	Sales
Quantity	Quantity
Discount	Discount
Profit	Profit

OK Cancel Help

OLE DB Destination Editor

Configure the properties used to insert data into a relational database using an OLE DB provider.

Connection Manager
Mappings
Error Output

Available Inp... Available Des...

Input Column	Destination Column
Row ID	Row ID
Order ID	Order ID
Order Date	Order Date
Ship Date	Ship Date
Ship Mode	Ship Mode
Customer ID	Customer ID
Product ID	Product ID
Sales	Sales
Quantity	Quantity
Discount	Discount
Profit	Profit

OK Cancel Help

Staging Row Counts After Load:

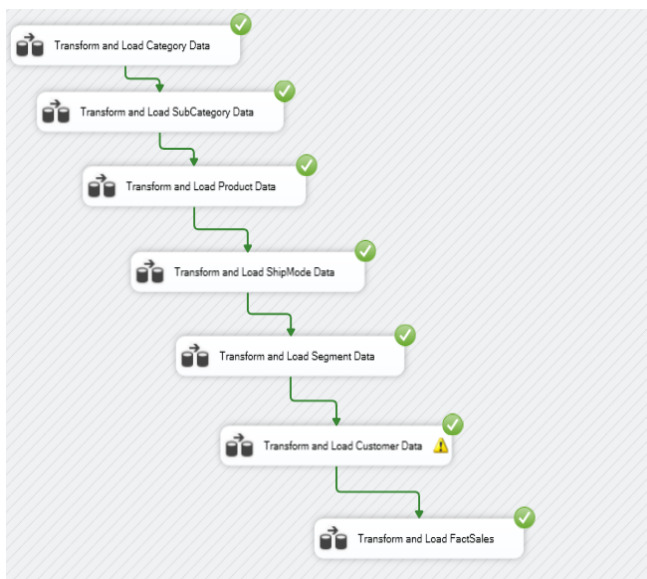
Staging Table	Source	Rows
StgCustomers	SQL Server DB	793
StgProducts	CSV File	1,894
StgOrders	Excel File	9,994
StgCustomerLocations	Text File	793

Package 2 — Superstore_Load_DW.dtsx

This package transforms and loads data from the staging database into the data warehouse. It contains 7 Data Flow Tasks executed in order based on foreign key dependencies.

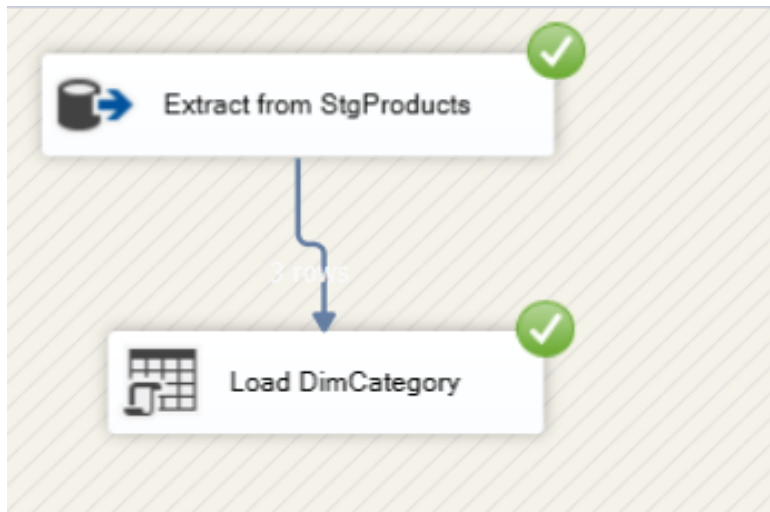
Execution Order (based on foreign key dependencies):

1. Transform and Load Category Data
2. Transform and Load SubCategory Data
3. Transform and Load Product Data
4. Transform and Load ShipMode Data
5. Transform and Load Segment Data
6. Transform and Load Customer Data
7. Transform and Load FactSales



Task 1 — Transform and Load Category Data

- Source: StgProducts (DISTINCT Category values)
- Stored Procedure: UpdateDimCategory
- Logic: If category doesn't exist → INSERT, else → UPDATE
- Rows loaded: 3

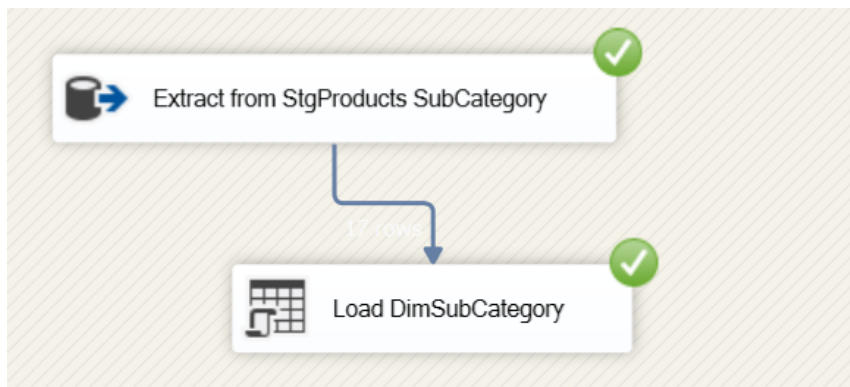


```
CREATE PROCEDURE dbo.UpdateDimCategory
@CategoryName NVARCHAR(50)
AS
BEGIN
    IF NOT EXISTS (
        SELECT CategorySK FROM dbo.DimCategory
        WHERE CategoryName = @CategoryName
    )
    BEGIN
        INSERT INTO dbo.DimCategory (AlternateCategoryID, CategoryName, InsertDate, ModifiedDate)
        VALUES (0, @CategoryName, GETDATE(), GETDATE())
    END
    ELSE
    BEGIN
        UPDATE dbo.DimCategory
        SET ModifiedDate = GETDATE()
        WHERE CategoryName = @CategoryName
    END
END

USE Superstore_DW;
SELECT * FROM dbo.DimCategory;
```

Task 2 — Transform and Load SubCategory Data

- Source: StgProducts (DISTINCT SubCategory + Category values)
- Stored Procedure: UpdateDimSubCategory
- Transformation: Looks up CategorySK from DimCategory using CategoryName — **surrogate key swap**
- Rows loaded: 17



```

CREATE PROCEDURE dbo.UpdateDimSubCategory
@SubCategoryName NVARCHAR(50),
@CategoryName NVARCHAR(50)
AS
BEGIN
    DECLARE @CategorySK INT

    SELECT @CategorySK = CategorySK
    FROM dbo.DimCategory
    WHERE CategoryName = @CategoryName

    IF NOT EXISTS (
        SELECT SubCategorySK FROM dbo.DimSubCategory
        WHERE SubCategoryName = @SubCategoryName
    )
    BEGIN
        INSERT INTO dbo.DimSubCategory
        (AlternateSubCategoryID, SubCategoryName, CategorySK, InsertDate, ModifiedDate)
        VALUES (0, @SubCategoryName, @CategorySK, GETDATE(), GETDATE())
    END
    ELSE
    BEGIN
        UPDATE dbo.DimSubCategory
        SET CategorySK = @CategorySK,
            ModifiedDate = GETDATE()
        WHERE SubCategoryName = @SubCategoryName
    END
END

USE Superstore_DW;
SELECT * FROM dbo.DimSubCategory;

```

Task 3 — Transform and Load Product Data

- Source: StgProducts
- Stored Procedure: UpdateDimProduct
- Transformation: Looks up SubCategorySK from DimSubCategory — **surrogate key swap**
- Rows loaded: 1,862



```

CREATE PROCEDURE dbo.UpdateDimProduct
@ProductID NVARCHAR(50),
@ProductName NVARCHAR(200),
@SubCategoryName NVARCHAR(50)
AS
BEGIN
    DECLARE @SubCategorySK INT

    SELECT @SubCategorySK = SubCategorySK
    FROM dbo.DimSubCategory
    WHERE SubCategoryName = @SubCategoryName

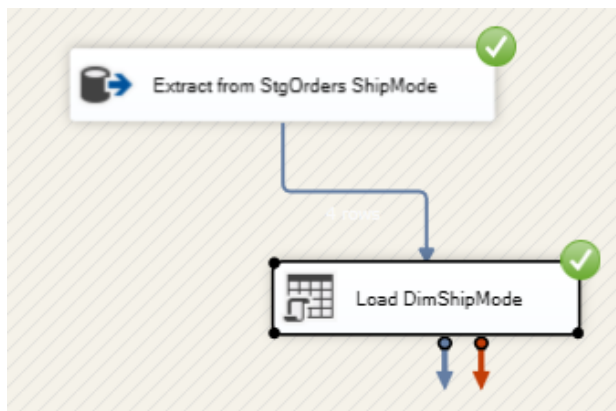
    IF NOT EXISTS (
        SELECT ProductSK FROM dbo.DimProduct
        WHERE AlternateProductID = @ProductID
    )
    BEGIN
        INSERT INTO dbo.DimProduct
        (AlternateProductID, ProductName, SubCategorySK, InsertDate, ModifiedDate)
        VALUES (@ProductID, @ProductName, @SubCategorySK, GETDATE(), GETDATE())
    END
    ELSE
    BEGIN
        UPDATE dbo.DimProduct
        SET ProductName = @ProductName,
            SubCategorySK = @SubCategorySK,
            ModifiedDate = GETDATE()
        WHERE AlternateProductID = @ProductID
    END
END

USE Superstore_DW;
SELECT COUNT(*) AS ProductCount FROM dbo.DimProduct;

```

Task 4 — Transform and Load ShipMode Data

- Source: StgOrders (DISTINCT Ship Mode values)
- Stored Procedure: UpdateDimShipMode
- Rows loaded: 4



```

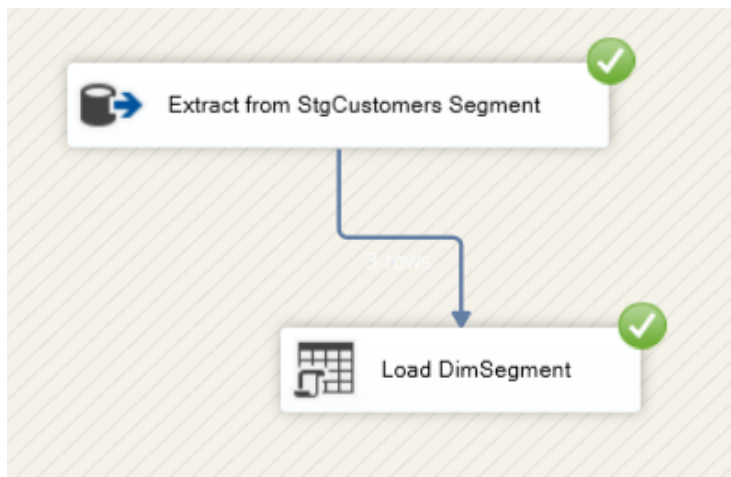
CREATE PROCEDURE dbo.UpdateDimShipMode
@ShipModeName NVARCHAR(50)
AS
BEGIN
    IF NOT EXISTS (
        SELECT ShipModeSK FROM dbo.DimShipMode
        WHERE ShipModeName = @ShipModeName
    )
    BEGIN
        INSERT INTO dbo.DimShipMode
        (ShipModeName, InsertDate, ModifiedDate)
        VALUES (@ShipModeName, GETDATE(), GETDATE())
    END
    ELSE
    BEGIN
        UPDATE dbo.DimShipMode
        SET ModifiedDate = GETDATE()
        WHERE ShipModeName = @ShipModeName
    END
END

USE Superstore_DW;
SELECT * FROM dbo.DimShipMode;

```

Task 5 — Transform and Load Segment Data

- Source: StgCustomers Segment (DISTINCT Segment values)
- Stored Procedure: UpdateDimSegment
- Rows loaded: 3



```

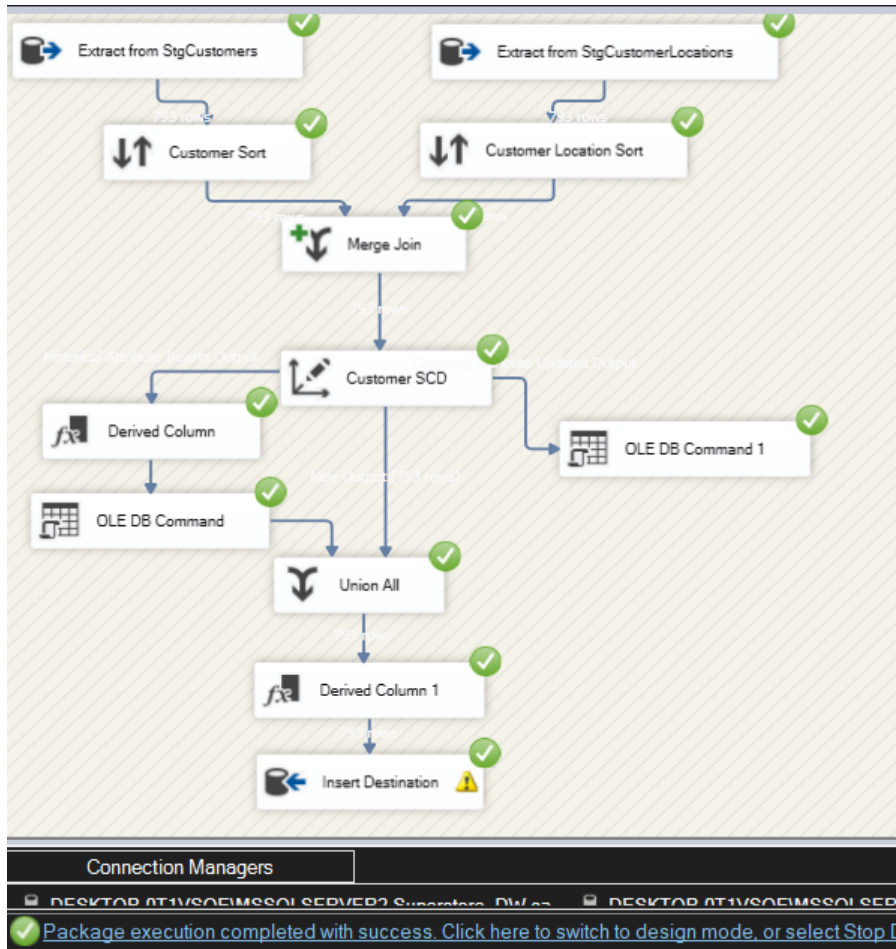
CREATE PROCEDURE dbo.UpdateDimSegment
@SegmentName NVARCHAR(50)
AS
BEGIN
    IF NOT EXISTS (
        SELECT SegmentSK FROM dbo.DimSegment
        WHERE SegmentName = @SegmentName
    )
    BEGIN
        INSERT INTO dbo.DimSegment
        (SegmentName, InsertDate, ModifiedDate)
        VALUES (@SegmentName, GETDATE(), GETDATE())
    END
    ELSE
    BEGIN
        UPDATE dbo.DimSegment
        SET ModifiedDate = GETDATE()
        WHERE SegmentName = @SegmentName
    END
END

USE Superstore_DW;
SELECT * FROM dbo.DimSegment;

```

Task 6 — Transform and Load Customer Data (SCD Type 2)

- Source 1: StgCustomers — customer demographics
- Source 2: StgCustomerLocations — customer address data
- Transformation: Two OLE DB Sources → Sort → Merge Join (Left Outer Join on CustomerID) → Derived Column (replace NULL Gender/Title) → Slowly Changing Dimension Wizard
- SCD Configuration:
 - Business Key: CustomerID → AlternateCustomerID
 - Changing attributes (Type 1): CustomerName — **overwrite on change**
 - Historical attributes (Type 2): City, State, PostalCode, Region, Country — **insert new row on change**
 - Start/End date columns: StartDate, EndDate
- Rows loaded: 793



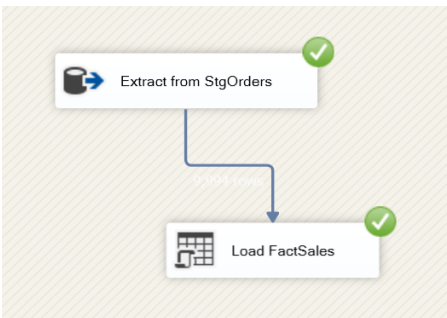
Task 7 — Transform and Load FactSales

- Source: StgOrders joined with StgCustomers (to get Segment)
- Stored Procedure: LoadFactSales
- Transformation: Looks up all Surrogate Keys from all dimension tables inside the stored procedure
- Measures loaded: Sales, Profit, Quantity, Discount
- accm_txn_create_time set to GETDATE() on insert
- Rows loaded: 9,986

DWBI ASSIGNMENT I – IT23302044

The screenshot displays the SQL Server Data Tools (SSIS) environment. On the left, the 'Superstore_tsx [Design]' window shows a data flow task with the following steps: Transform and Load Category Data, Transform and Load SubCategory Data, Transform and Load Product Data, Transform and Load ShipMode Data, Transform and Load Segment Data, Transform and Load Customer Data, and Transform and Load FactSales. The task is marked as 'Package execution completed with success'. On the right, the 'DWBI_Script...W (sa (72))' window shows a SQL query that selects data from 'dbo.StgOrders' and 'dbo.FactSales' into 'Superstore_DW'. The query includes a 'SELECT COUNT(*) AS FactSalesCount' and a 'SELECT TOP 5' clause. The results window shows the following data:

SalesSK	OrderID	CustomerSK	ProductSK	OrderDateSK	Sales	Profit	accc_txn_create_time
1	CA-2017-164042	2814	414	20170523	18.32	-46.71	2026-03-25 08:18:13.947
2	CA-2017-164042	2814	1366	20170523	48.81	-11.56	2026-03-25 08:18:13.987
3	CA-2017-164042	2814	717	20170523	1.16	-1.96	2026-03-25 08:18:14.000
4	CA-2014-132664	2819	1492	20141202	119.80	29.95	2026-03-25 08:18:14.013
5	US-2014-195817	3078	1304	20141003	61.56	4.61	2026-03-25 08:18:14.030



The screenshot shows the 'Advanced Editor for Load FactSales' window. It provides access to the low-level properties of the data flow component. The 'Component Properties' tab is selected, showing the 'Available Input' and 'Available Destination' columns. The 'Input Column' and 'Destination Column' tables are shown below.

Input Column	Destination Column
Order ID	@OrderID
Customer ID	@CustomerID
Product ID	@ProductID
Order Date	@OrderDate
Ship Date	@ShipDate
Ship Mode	@ShipModeName
Segment	@SegmentName
Sales	@Sales
Quantity	@Quantity
Discount	@Discount
Profit	@Profit

```

CREATE PROCEDURE dbo.LoadFactSales
@OrderID NVARCHAR(20),
@CustomerID NVARCHAR(20),
@ProductID NVARCHAR(50),
@OrderDate DATETIME,
@ShipDate DATETIME,
@ShipModeName NVARCHAR(50),
@SegmentName NVARCHAR(50),
@Sales DECIMAL(10,2),
@Quantity INT,
@Discount DECIMAL(5,2),
@Profit DECIMAL(10,2)
AS
BEGIN
    DECLARE @CustomerSK INT
    DECLARE @ProductSK INT
    DECLARE @OrderDateSK INT
    DECLARE @ShipDateSK INT
    DECLARE @ShipModeSK INT
    DECLARE @SegmentSK INT

    -- Lookup Surrogate Keys
    SELECT @CustomerSK = CustomerSK
    FROM dbo.DimCustomer
    WHERE AlternateCustomerID = @CustomerID
    AND (EndDate IS NULL OR IsCurrent = 1)

    SELECT @ProductSK = ProductSK
    FROM dbo.DimProduct
    WHERE AlternateProductID = @ProductID

    SELECT @OrderDateSK = DateKey
    FROM dbo.DimDate
    WHERE CAST(Date AS DATE) = CAST(@OrderDate AS DATE)

    SELECT @ShipModeSK = ShipModeSK
    FROM dbo.DimShipMode
    WHERE ShipModeName = @ShipModeName

    SELECT @SegmentSK = SegmentSK
    FROM dbo.DimSegment
    WHERE SegmentName = @SegmentName

    -- Insert into FactSales
    IF NOT EXISTS (
        SELECT SalesSK FROM dbo.FactSales
        WHERE OrderID = @OrderID
        AND ProductSK = @ProductSK
    )
    BEGIN
        INSERT INTO dbo.FactSales (
            OrderID, CustomerSK, ProductSK,
            OrderDateSK, ShipDateSK, ShipModeSK, SegmentSK,
            Sales, Quantity, Discount, Profit,
            accm_txn_create_time,
            InsertDate, ModifiedDate
        )
        VALUES (
            @OrderID, @CustomerSK, @ProductSK,
            @OrderDateSK, @ShipDateSK, @ShipModeSK, @SegmentSK,
            @Sales, @Quantity, @Discount, @Profit,
            GETDATE(),
            GETDATE(), GETDATE()
        )
    END
END

```

Final DW Row Counts:

Table	Rows
DimCategory	3
DimSubCategory	17
DimProduct	1,862
DimShipMode	4
DimSegment	3
DimCustomer	793
FactSales	9,986

The screenshot shows a SQL Server Enterprise Manager window with a query titled 'DWBI_Script...W (sa (71))'. The query is executed successfully, and the results are displayed in a grid below the query editor.

```

USE Superstore_DW;

SELECT 'DimCustomer' AS TableName, COUNT(*) AS 'RowCount' FROM dbo.DimCustomer
UNION ALL
SELECT 'FactSales', COUNT(*) FROM dbo.FactSales;

USE Superstore_DW;

SELECT 'DimCategory' AS TableName, COUNT(*) AS 'RowCount' FROM dbo.DimCategory
UNION ALL
SELECT 'DimSubCategory', COUNT(*) FROM dbo.DimSubCategory
UNION ALL
SELECT 'DimProduct', COUNT(*) FROM dbo.DimProduct
UNION ALL
SELECT 'DimShipMode', COUNT(*) FROM dbo.DimShipMode
UNION ALL
SELECT 'DimSegment', COUNT(*) FROM dbo.DimSegment
UNION ALL
SELECT 'DimCustomer', COUNT(*) FROM dbo.DimCustomer
UNION ALL
SELECT 'FactSales', COUNT(*) FROM dbo.FactSales;

```

TableName	RowCount
1 DimCategory	3
2 DimSubCategory	17
3 DimProduct	1862
4 DimShipMode	4
5 DimSegment	3
6 DimCustomer	793
7 FactSales	9986

Query executed successfully. DESKTOP-OT1V5QE\MSSQLSERVER... sa (71) Superstore_DW 00:00:00 Row: 1, Col: 1 7 rows

Task 6 — Accumulating Fact Table

ETL Development — Accumulating Fact Tables

Overview

The `FactSales` table was extended with accumulating fact columns to track the lifecycle of each sales transaction from creation to completion. This pattern is used in data warehousing to measure process performance — specifically how long it takes for an order to be completed after it is placed.

Additional Columns Added to FactSales

Column	Value	Description
<code>accm_txn_create_time</code>	2026-03-25 17:01	When order was loaded into DW
<code>accm_txn_complete_time</code>	2026-03-27	When order was completed
<code>txn_process_time_hours</code>	31	Hours between creation and completion
Records Updated	12	Orders with completion time populated

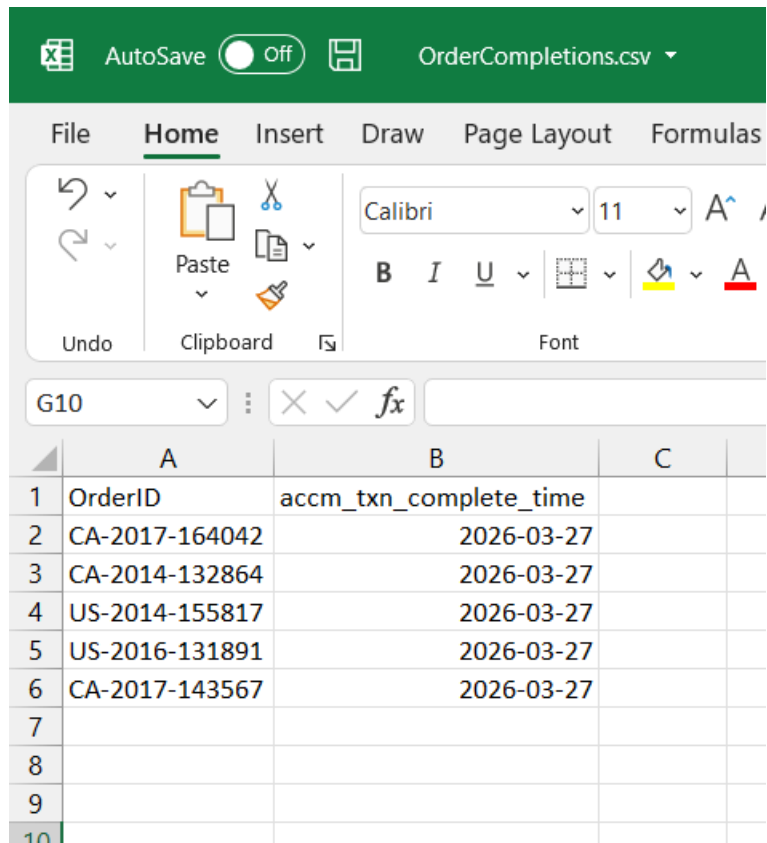
The `txn_process_time_hours` is a **computed column** in SQL Server — it recalculates automatically whenever `accm_txn_complete_time` is updated, requiring no manual calculation!

The screenshot shows a SQL Server query window with the following query: `SELECT * FROM dbo.FactSales;`. The results pane displays a table with 14 columns: `OrderID`, `CustomerSK`, `ProductSK`, `OrderDateSK`, `ShipDateSK`, `ShipModeSK`, `SegmentSK`, `Sales`, `Quantity`, `Discount`, `Profit`, `accm_txn_create_time`, `accm_txn_complete_time`, and `txn_process_time_hours`. The data shows several rows of sales transactions with their respective completion times and process durations.

Completion Times Dataset

A separate CSV file `OrderCompletions.csv` was prepared containing order IDs and their completion timestamps. This simulates a real-world scenario where an external system reports when transactions are completed.

Column	Description
<code>OrderID</code>	Natural key matching <code>OrderID</code> in <code>FactSales</code>
<code>accm_txn_complete_time</code>	The datetime when the transaction was completed

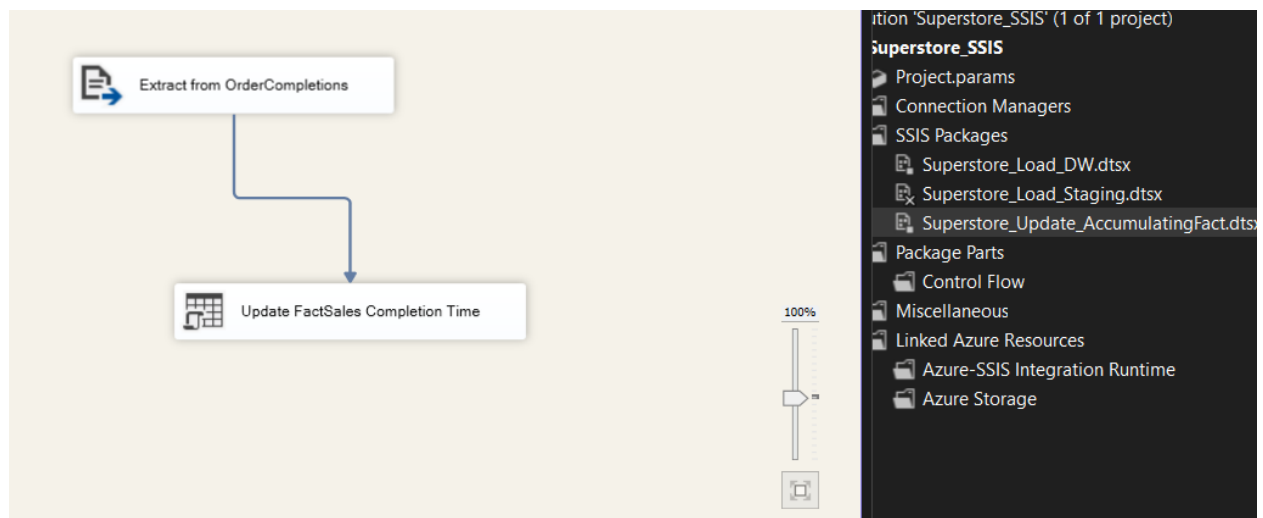


	A	B	C
1	OrderID	accm_txn_complete_time	
2	CA-2017-164042	2026-03-27	
3	CA-2014-132864	2026-03-27	
4	US-2014-155817	2026-03-27	
5	US-2016-131891	2026-03-27	
6	CA-2017-143567	2026-03-27	
7			
8			
9			
10			

SSIS Package — Superstore_Update_AccumulatingFact.dtsx

A separate SSIS package was developed to read from `OrderCompletions.csv` and update the corresponding records in `FactSales`.

Package Design:



Stored Procedure — UpdateAccumulatingFact:

```

USE Superstore_DW;

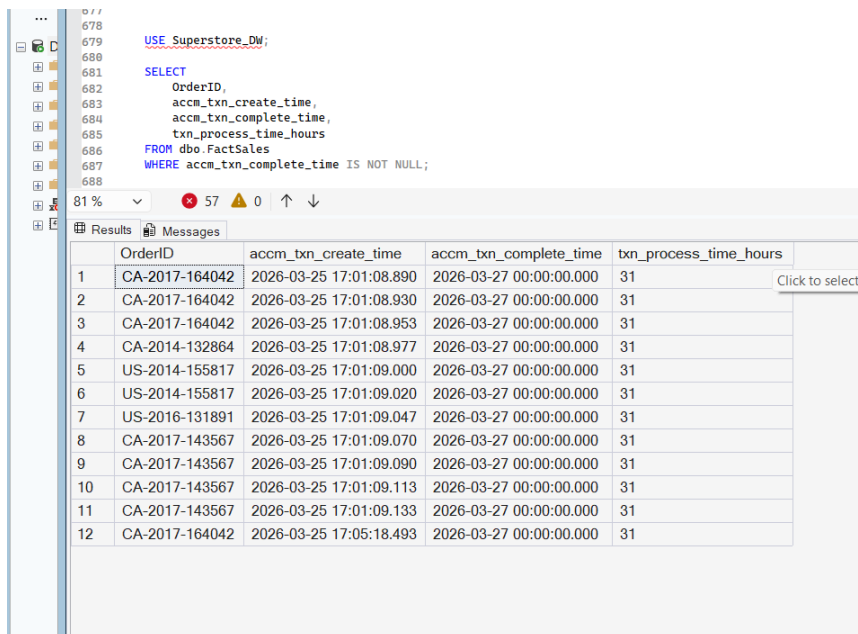
CREATE PROCEDURE dbo.UpdateAccumulatingFact
@OrderID NVARCHAR(20),
@AccmTxnCompleteTime NVARCHAR(50)
BEGIN
    UPDATE dbo.FactSales
    SET
        accm_txn_complete_time = @AccmTxnCompleteTime,
        ModifiedDate = GETDATE()
    WHERE OrderID = @OrderID
    AND accm_txn_complete_time IS NULL
END

```

The stored procedure only updates records where `accm_txn_complete_time IS NULL` — this prevents overwriting already completed transactions if the package is run multiple times!

Verification

After running the accumulating fact update package, the results were verified in SSMS:



	OrderID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	CA-2017-164042	2026-03-25 17:01:08.890	2026-03-27 00:00:00.000	31
2	CA-2017-164042	2026-03-25 17:01:08.930	2026-03-27 00:00:00.000	31
3	CA-2017-164042	2026-03-25 17:01:08.953	2026-03-27 00:00:00.000	31
4	CA-2014-132864	2026-03-25 17:01:08.977	2026-03-27 00:00:00.000	31
5	US-2014-155817	2026-03-25 17:01:09.000	2026-03-27 00:00:00.000	31
6	US-2014-155817	2026-03-25 17:01:09.020	2026-03-27 00:00:00.000	31
7	US-2016-131891	2026-03-25 17:01:09.047	2026-03-27 00:00:00.000	31
8	CA-2017-143567	2026-03-25 17:01:09.070	2026-03-27 00:00:00.000	31
9	CA-2017-143567	2026-03-25 17:01:09.090	2026-03-27 00:00:00.000	31
10	CA-2017-143567	2026-03-25 17:01:09.113	2026-03-27 00:00:00.000	31
11	CA-2017-143567	2026-03-25 17:01:09.133	2026-03-27 00:00:00.000	31
12	CA-2017-164042	2026-03-25 17:05:18.493	2026-03-27 00:00:00.000	31

The query returned 12 records showing:

- `accm_txn_create_time` — when the order was loaded (2026-03-25)
- `accm_txn_complete_time` — when the order was completed (2026-03-27)
- `txn_process_time_hours` — **31 hours** (automatically computed!)